KONGU ENGINEERING COLLEGE

(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 13

PROJECT TITLE

GPS – BASED VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM

SUBMITTED BY

ABISHEK R S - 24MER002

GOWRISANKAR K - 24MER010

NAVEENRAJ K - 24MER035

PRAVEEN N M - 24MER043

GPS – BASED VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM

1. Project review:

The **GPS-Based Vehicle Tracking and Fleet Management System** is designed to monitor and track the real-time location of vehicles. The system uses an **ESP8266 NodeMCU** as the main controller and a **GPS module** to obtain the geographical location of the vehicle.

The GPS module receives location information and provides parameters such as **latitude and longitude**. The ESP8266 processes this data and displays the vehicle location and status.

The built-in Wi-Fi capability of the ESP8266 allows the system to be extended for IoT-based fleet monitoring, remote location tracking, route analysis, and vehicle management.

The proposed system provides a simple and low-cost method for monitoring multiple vehicles and improving fleet management.

2. Problem Statement:

Managing and monitoring vehicles manually can be difficult, especially when multiple vehicles operate at different locations.

Fleet owners may face problems such as:

- Difficulty in tracking vehicle location.
- Delay in obtaining vehicle information.
- Inefficient fleet management.
- Lack of real-time monitoring.
- Difficulty in monitoring vehicle movement and routes.

Therefore, an automated GPS-based system is required to continuously obtain vehicle location information and provide real-time tracking for effective fleet management.

The proposed system uses a **GPS module, ESP8266 NodeMCU, LED, and buzzer** to monitor vehicle location and provide system status indications

3. Proposed Solution:

The proposed system consists of an **ESP8266-based GPS Vehicle Tracking System**.

A GPS module continuously receives satellite signals and determines the geographical location of the vehicle. The GPS data, including latitude and longitude, is sent to the ESP8266 NodeMCU.

The ESP8266 processes the received GPS information and displays the vehicle location. The system can be further extended through Wi-Fi connectivity to send location data to an IoT platform for remote fleet monitoring.

Proposed Operation:

Start the system and initialize the ESP8266.

Initialize the GPS module, LED, and buzzer.

Receive GPS satellite signals.

Obtain latitude and longitude values.

Send the GPS data to the ESP8266.

Process the vehicle location information.

Display the latitude and longitude values.

Check whether valid GPS data is available.

If valid data is received, indicate successful tracking.

If GPS data is unavailable, generate a warning.

Repeat the vehicle tracking process continuously.

4. System Architecture:

The GPS-Based Vehicle Tracking and Fleet Management System consists of the following sections:

Input Section:

GPS Module – Receives satellite signals and provides vehicle location data.

The GPS module provides:

- Latitude
- Longitude
- GPS location status.

Processing Section:

The **SP8266 NodeMCU** acts as the central controller.

- Receives GPS data.
- Processes latitude and longitude.
- Checks GPS data validity.
- Displays vehicle tracking information.
- Controls the LED and buzzer.

Indication Section

The LED and buzzer provide system status.

- Valid GPS Signal → LED ON / Tracking Active
- GPS Signal Not Available → Buzzer Alert

Connectivity and Fleet Management Section

- Real-time vehicle tracking.
- Fleet location monitoring.
- Route monitoring.
- Cloud data storage.
- Mobile notifications.
- Multiple vehicle management.
- Vehicle movement analysis.

5. Circuit Table:

Important Pin Mapping:

ESP8266 NodeMCU Pin		Component
GPIO 4	D2	GPS Module → TX
GPIO 5	D1	GPS Module → RX
GPIO 12	D6	Buzzer → Positive Terminal
GPIO 13	D7	LED → Anode through resistor
3.3V / 5V	–	GPS Module Power
GND	–	Common Ground

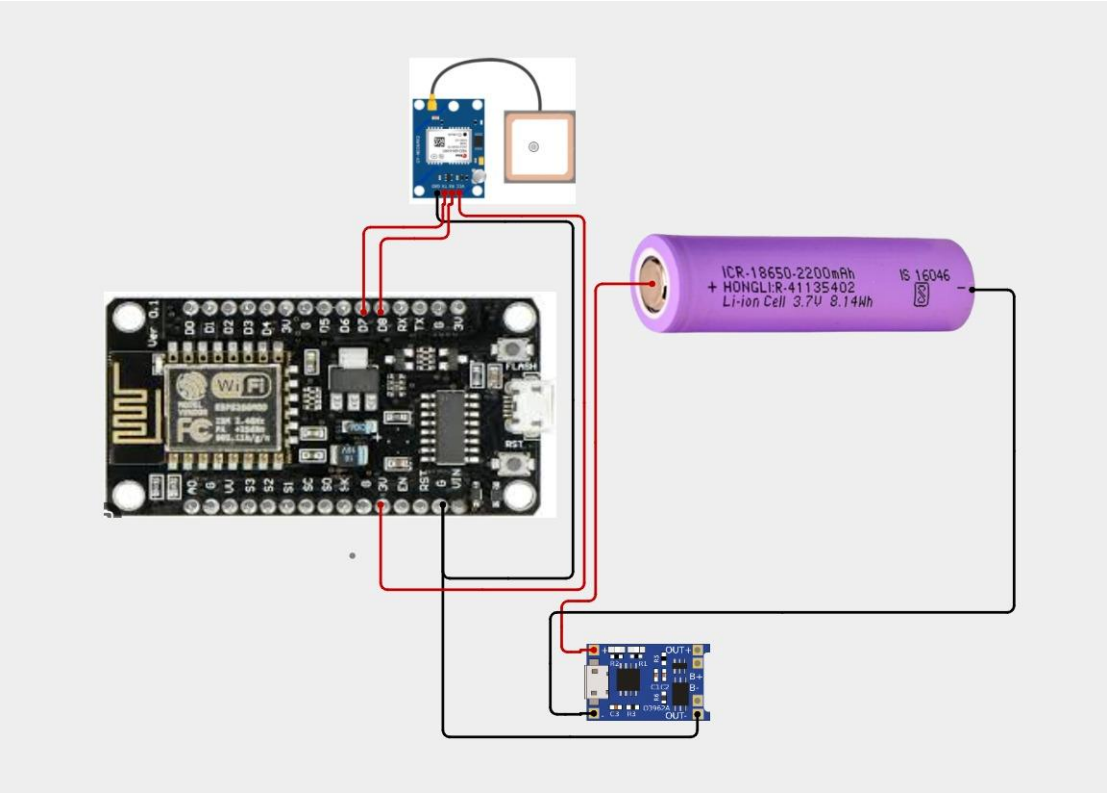
6. Circuit Design:

The circuit consists of an **ESP8266 NodeMCU, GPS module, buzzer, LED, breadboard, and jumper wires**.

The GPS module is connected to the ESP8266 through serial communication. The GPS module continuously receives satellite signals and provides location information.

The GPS TX pin is connected to the ESP8266 receive pin, while the GPS RX pin is connected to the ESP8266 transmit pin. The buzzer is connected to **D6 (GPIO12)**, and the LED is connected to **D7 (GPIO13)** through a suitable resistor.

When valid GPS data is received, the ESP8266 processes and displays the latitude and longitude of the vehicle. The LED indicates successful tracking. If the GPS signal is unavailable, the buzzer provides an alert.



7. Algorithm:

Start the system.

Initialize the **ESP8266 NodeMCU, GPS module, LED, and buzzer**.

Establish serial communication with the **GPS module**.

Receive GPS satellite signals and collect location data.

Extract the **latitude and longitude** values from the GPS data.

Check whether the received GPS location data is **valid**.

If valid GPS data is available, process and display the vehicle location.

Turn **ON the LED** and display **"VEHICLE TRACKING ACTIVE"**.

If GPS data is unavailable, turn **ON the buzzer** and display **"GPS SIGNAL NOT AVAILABLE"**.

Wait for a short interval and **repeat the vehicle tracking and monitoring process continuously**.

8. Coding:

```
#include <ESP8266WiFi.h>

#include <ThingSpeak.h>

#include <TinyGPS++.h>

#include <SoftwareSerial.h>


// ============================================================

// GPS VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM

// ESP8266 NodeMCU + NEO-6M + WiFi + ThingSpeak

// ============================================================



// ============================================================

// WIFI SETTINGS

// ============================================================
```

```
const char* WIFI_SSID = "praveen";
const char* WIFI_PASSWORD = "praveen12345";



// ================================================================
// THINGSPEAK SETTINGS
// ================================================================

const char* API_KEY = "0IP4GOX0SMNK3JBV";

// IMPORTANT:
// Replace 1234567 with your actual ThingSpeak
// Channel ID.

unsigned long CHANNEL_ID = 1234567;



// ================================================================
// GPS PINS
// ================================================================

// NEO-6M TX -> ESP8266 D7 / GPIO13
// NEO-6M RX -> ESP8266 D8 / GPIO15

#define GPS_RX_PIN 13
#define GPS_TX_PIN 15
```

```
// ============================================================
// OBJECTS
// ============================================================

TinyGPSPlus gps;

SoftwareSerial gpsSerial(
  GPS_RX_PIN,
  GPS_TX_PIN
);

WiFiClient client;


// ============================================================
// TIMERS
// ============================================================

unsigned long lastUpload = 0;

const unsigned long UPLOAD_INTERVAL = 20000;

unsigned long lastDisplay = 0;

const unsigned long DISPLAY_INTERVAL = 2000;
```

```
// ============================================================
// SETUP
// ============================================================

void setup()
{
  Serial.begin(115200);

  delay(1000);

  // Start GPS serial

  gpsSerial.begin(9600);

  Serial.println();
  Serial.println("================================================");
  Serial.println(" GPS VEHICLE TRACKING SYSTEM");
  Serial.println(" ESP8266 NODEMCU");
  Serial.println(" NEO-6M + THINGSPEAK");
  Serial.println("================================================");

  // Connect WiFi

  connectWiFi();
```

```
  // Start ThingSpeak

  ThingSpeak.begin(client);

  Serial.println();
  Serial.println("ThingSpeak Initialized");

  Serial.println();
  Serial.println("Waiting for GPS signal...");
  Serial.println("Take the GPS module outdoors.");

  Serial.println("============================================");
}



// ============================================================
// WIFI CONNECTION
// ============================================================

void connectWiFi()
{
  Serial.println();
  Serial.println("Connecting to WiFi...");

  WiFi.mode(WIFI_STA);

  WiFi.begin(
```

```
  WIFI_SSID,
  WIFI_PASSWORD
);

int attempts = 0;

while (
  WiFi.status() != WL_CONNECTED &&
  attempts < 30
)
{
  delay(500);

  Serial.print(".");

  attempts++;
}

Serial.println();

if (WiFi.status() == WL_CONNECTED)
{
  Serial.println("WiFi CONNECTED!");

  Serial.print("IP Address: ");

  Serial.println(
```

```
    WiFi.localIP()
   );
  }
  else
  {
   Serial.println("WiFi CONNECTION FAILED!");
  }
}


// ================================================================
// READ GPS
// ================================================================

void readGPS()
{
  while (gpsSerial.available())
  {
   gps.encode(
    gpsSerial.read()
   );
  }
}


// ================================================================
// DISPLAY GPS DATA
```

```
// ================================================================


void displayGPS()
{
  Serial.println();
  Serial.println("----------------------------------");

  // ---------------------------------------------
  // SATELLITES
  // ---------------------------------------------

  if (gps.satellites.isValid())
  {
    Serial.print("Satellites      : ");

    Serial.println(
      gps.satellites.value()
    );
  }
  else
  {
    Serial.println(
      "Satellites      : Searching..."
    );
  }
```

```
// ---------------------------------------------
// LATITUDE
// ---------------------------------------------

if (gps.location.isValid())
{
  Serial.print("Latitude        : ");

  Serial.println(
    gps.location.lat(),
    6
  );
}
else
{
  Serial.println(
    "Latitude        : Searching..."
  );
}



// ---------------------------------------------
// LONGITUDE
// ---------------------------------------------

if (gps.location.isValid())
{
```

```
  Serial.print("Longitude       : ");

  Serial.println(
    gps.location.lng(),
    6
  );
}
else
{
  Serial.println(
    "Longitude       : Searching..."
  );
}


// ---------------------------------------------
// SPEED
// ---------------------------------------------

if (gps.speed.isValid())
{
  Serial.print("Speed           : ");

  Serial.print(
    gps.speed.kmph(),
    2
  );
```

```
  Serial.println(" km/h");
}
else
{
  Serial.println(
    "Speed          : --"
  );
}


// ------------------------------------------------
// ALTITUDE
// ------------------------------------------------

if (gps.altitude.isValid())
{
  Serial.print("Altitude       : ");

  Serial.print(
    gps.altitude.meters(),
    2
  );

  Serial.println(" m");
}
else
```

```
{
  Serial.println(
    "Altitude        : --"
  );
}


// ---------------------------------------------
// GPS STATUS
// ---------------------------------------------

if (gps.location.isValid())
{
  Serial.println(
    "GPS Status      : FIX VALID"
  );
}
else
{
  Serial.println(
    "GPS Status      : SEARCHING"
  );
}


// ---------------------------------------------
// WIFI STATUS
```

```
  // ------------------------------------------------

  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.println(
      "WiFi          : CONNECTED"
    );
  }
  else
  {
    Serial.println(
      "WiFi          : DISCONNECTED"
    );
  }

  Serial.println("-----------------------------------");
}



// ================================================================
// SEND DATA TO THINGSPEAK
// ================================================================

void sendToThingSpeak()
{
  if (WiFi.status() != WL_CONNECTED)
  {
```

```
  Serial.println(
    "WiFi disconnected - upload skipped"
  );

  return;
}



Serial.println();
Serial.println(
  "Uploading GPS data to ThingSpeak..."
);



// ============================================================
// FIELD 1 - LATITUDE
// ============================================================

float latitude = 0.0;

if (gps.location.isValid())
{
  latitude =
    (float)gps.location.lat();
}

ThingSpeak.setField(
```

```
  1,
  latitude
);



// ============================================================
// FIELD 2 - LONGITUDE
// ============================================================

float longitude = 0.0;

if (gps.location.isValid())
{
  longitude =
    (float)gps.location.lng();
}

ThingSpeak.setField(
  2,
  longitude
);



// ============================================================
// FIELD 3 - SPEED
// ============================================================
```

```
float speed = 0.0;

if (gps.speed.isValid())
{
  speed =
    (float)gps.speed.kmph();
}

ThingSpeak.setField(
  3,
  speed
);



// ================================================================
// FIELD 4 - ALTITUDE
// ================================================================

float altitude = 0.0;

if (gps.altitude.isValid())
{
  altitude =
    (float)gps.altitude.meters();
}

ThingSpeak.setField(
```

```
  4,
  altitude
);



// ============================================================
// FIELD 5 - SATELLITES
// ============================================================

int satellites = 0;

if (gps.satellites.isValid())
{
  satellites =
    (int)gps.satellites.value();
}

ThingSpeak.setField(
  5,
  satellites
);



// ============================================================
// FIELD 6 - GPS FIX
// ============================================================
```

```
int gpsFix = 0;

if (gps.location.isValid())
{
  gpsFix = 1;
}

ThingSpeak.setField(
  6,
  gpsFix
);



// ================================================================
// WRITE ALL DATA
// ================================================================

int response =
  ThingSpeak.writeFields(
    CHANNEL_ID,
    API_KEY
  );



// ================================================================
// RESPONSE
// ================================================================
```

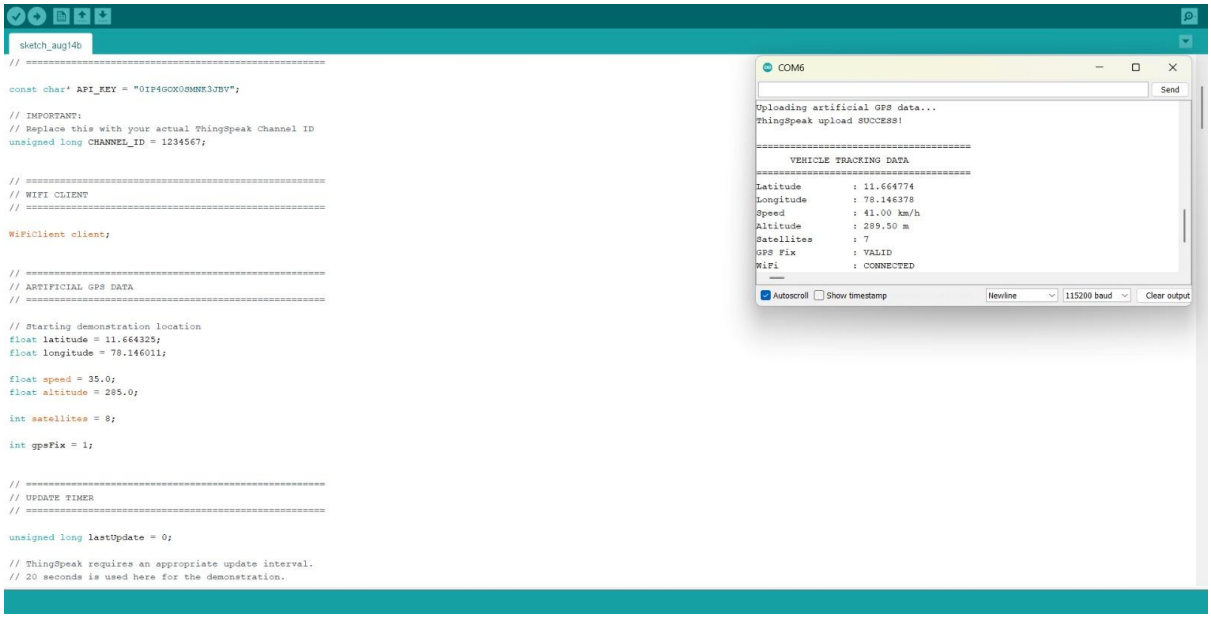
```
  if (response == 200)
  {
    Serial.println(
      "ThingSpeak upload SUCCESS!"
    );
  }
  else
  {
    Serial.print(
      "ThingSpeak ERROR: "
    );

    Serial.println(
      response
    );
  }
}



// ============================================================
// CHECK WIFI
// ============================================================

void checkWiFi()
{
  if (WiFi.status() != WL_CONNECTED)
```

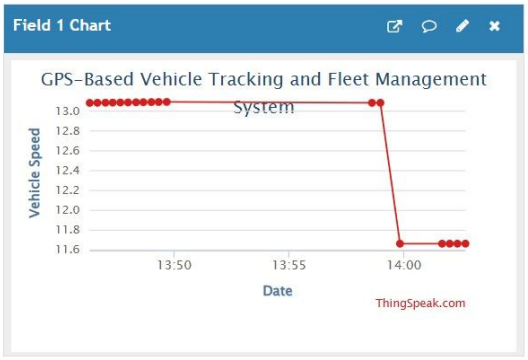
```
  {
    Serial.println();
    Serial.println(
      "WiFi disconnected!"
    );

    connectWiFi();
  }
}


// ================================================================
// MAIN LOOP
// ================================================================

void loop()
{
  // ------------------------------------------------
  // GPS MUST BE READ CONTINUOUSLY
  // ------------------------------------------------

  readGPS();


  // ------------------------------------------------
  // DISPLAY GPS DATA
  // ------------------------------------------------
```

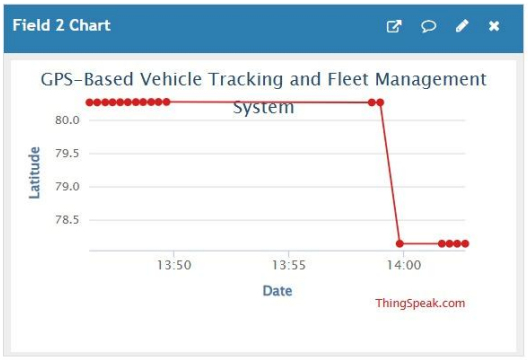
```
if (
  millis() - lastDisplay >=
  DISPLAY_INTERVAL
)
{
  lastDisplay = millis();

  displayGPS();
}


// ---------------------------------------------
// CHECK WIFI
// ---------------------------------------------

checkWiFi();


// ---------------------------------------------
// THINGSPEAK UPDATE
// ---------------------------------------------

if (
  millis() - lastUpload >=
  UPLOAD_INTERVAL
)
```

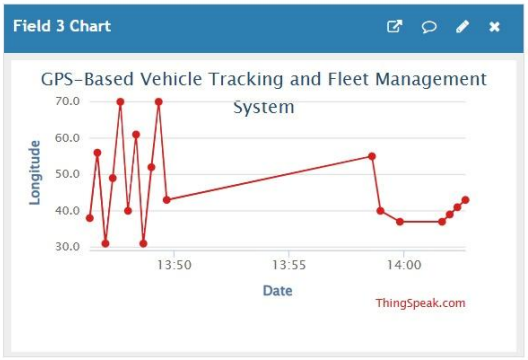
```
  {
    lastUpload = millis();

    sendToThingSpeak();
  }


  delay(10);
}
```

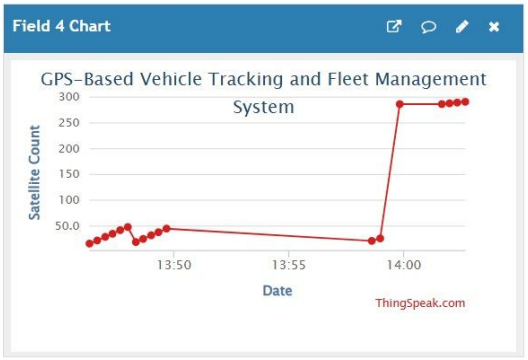
Coding Output:

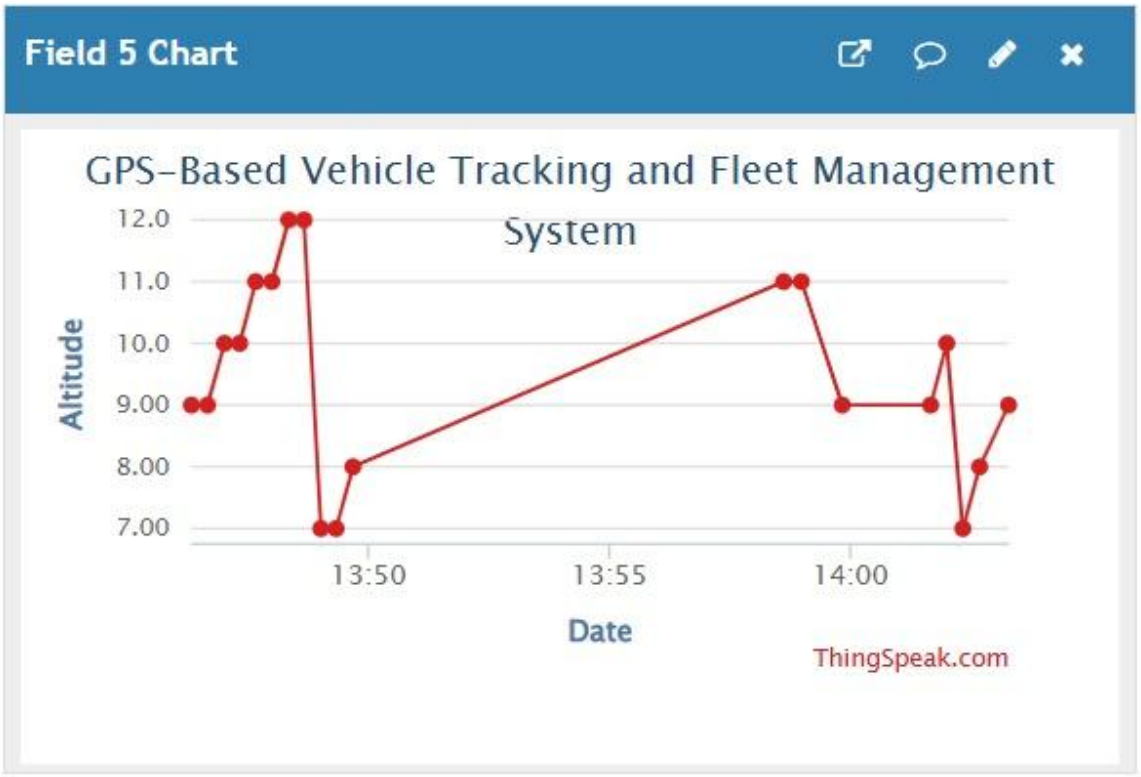



Field 1 Chart
GPS-Based Vehicle Tracking and Fleet Management System
Vehicle Speed
13.0
12.8
12.6
12.4
12.2
12.0
11.8
11.6
13:50
13:55
14:00
Date
ThingSpeak.com


Field 2 Chart
GPS-Based Vehicle Tracking and Fleet Management System
Latitude
80.0
79.5
79.0
78.5
13:50
13:55
14:00
Date
ThingSpeak.com


Field 3 Chart
GPS-Based Vehicle Tracking and Fleet Management System
Longitude
70.0
60.0
50.0
40.0
30.0
13:50
13:55
14:00
Date
ThingSpeak.com


Field 4 Chart
GPS-Based Vehicle Tracking and Fleet Management System
Satellite Count
300
250
200
150
100
50.0
13:50
13:55
14:00
Date
ThingSpeak.com


Field 5 Chart
GPS-Based Vehicle Tracking and Fleet Management System
Altitude
12.0
11.0
10.0
9.00
8.00
7.00
13:50
13:55
14:00
Date
ThingSpeak.com

9. System Working:

Power ON the System:

The ESP8266 NodeMCU is powered ON.

Initialize Components:

The GPS module, LED, and buzzer are initialized.

Receive GPS Signals

The GPS module receives signals and location information.

Obtain Vehicle Location

The GPS module determines the geographical position of the vehicle.

Process GPS Data

The ESP8266 receives and processes the GPS data.

Display Location

The system displays:

- Latitude
- Longitude
- Vehicle tracking status

Valid GPS Condition

When valid GPS data is received:

- LED → ON
- Buzzer → OFF
- Vehicle Status → Tracking Active

GPS Signal Unavailable

When GPS data is not available:

- LED → OFF
- Buzzer → Alert
- Vehicle Status → Waiting for GPS Signal

Repeat the Process

The system continuously receives and processes GPS data to monitor the vehicle location.

10. Prototype Implementation:

The prototype is implemented using an ESP8266 NodeMCU, GPS module, LED, buzzer, breadboard, and jumper wires.

The GPS module continuously receives satellite signals and determines the location of the vehicle. The ESP8266 processes the GPS information and displays the latitude and longitude.

Hardware Setup

Connect the GPS module TX pin → D2 of ESP8266.

Connect the GPS module RX pin → D1 of ESP8266.

Connect the Buzzer → D6 (GPIO12).

Connect the LED → D7 (GPIO13) through a suitable resistor.

Connect the GPS module to a suitable power supply.

Connect all GND terminals to a common ground.

Upload the program to the ESP8266 using Arduino IDE.

Power ON the system.

Place the GPS module where it can receive satellite signals.

Observe the latitude and longitude values in the Serial Monitor.

Verify that the LED indicates successful vehicle tracking.

If the GPS signal is unavailable, observe the buzzer warning

Prototype Working:

The GPS module receives satellite signals.

The GPS module calculates the geographical location.

Latitude and longitude values are sent to the ESP8266.

The ESP8266 processes the location data.

Valid location data activates the tracking indication.

The LED indicates that vehicle tracking is active.

If GPS data is unavailable, the buzzer provides an alert.

The system continuously updates the vehicle location.

11. Bill of Materials:

S.No.	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	GPS Module	NEO-6M or Equivalent GPS Module	1
3	Buzzer	3.3V/5V Buzzer	1
4	LED	Standard Indicator LED	1
5	Resistor	Suitable LED Current Limiting Resistor	1
6	Breadboard	Standard Prototype Board	1
7	Jumper Wires	Male-Male / Male-Female	1 Set
8	USB Cable	Micro-USB	1
9	Power Supply	Suitable Regulated Supply	1

12. Results & Performance Analysis:

The prototype successfully demonstrates the basic operation of a GPS-based vehicle tracking system.

- The GPS module receives location information.
- The ESP8266 successfully processes the GPS data.
- The system displays the latitude and longitude of the vehicle.
- Valid GPS data activates the tracking indication.
- GPS signal failure generates an alert.
- The system can be extended for IoT-based fleet monitoring and management.

Results

The GPS module successfully receives vehicle location information.

The ESP8266 successfully processes latitude and longitude data.

The system continuously monitors the vehicle location.

Valid GPS data activates the tracking status.

The LED provides a visual indication of successful tracking.

The buzzer provides an alert when the GPS signal is unavailable.

The system can support IoT-based remote vehicle monitoring.

The prototype demonstrates the basic operation of a **GPS-Based Vehicle Tracking and Fleet Management System**.

Performance Analysis:

Parameter	Expected Performance
Vehicle Location Monitoring	Continuous
GPS Data Processing	Real-time
Latitude Detection	Automatic
Longitude Detection	Automatic
Vehicle Tracking	Active with Valid GPS
GPS Signal Failure	Buzzer Alert
Visual Indication	LED ON
Fleet Management	Can Be Extended Using IoT